Aix-Marseille Université

Traitement et Analyse d'Images

Rapport de Travaux Pratiques

Détection et Évaluation Quantitative de Contours

Développement d'une Application Interactive OpenCV C++
et Évaluation Comparative sur le Benchmark RUG (40 Images)

Auteur :

Youssef BENABBOU
Étudiant en Master Informatique
Aix-Marseille Université

Enseignant Responsable :

Pr. Sébastien MAVROMATIS
Enseignant-Chercheur
Aix-Marseille Université

Application interactive en C++ / OpenCV 4.x
Visualisation en 4 cadrans synchronisés, curseurs dynamiques, métriques temps réel (P, TFP, TFN),
tolérance spatiale de voisinage 3×3 et banc d'essai par lot automatique sur 40 images.

Année Universitaire 2025 – 2026

Rendu : Octobre 2026

Résumé

Ce rapport détaille le travail réalisé dans le cadre du TP3 de Traitement et Analyse d'Images, consacré à l'implémentation logicielle et à l'évaluation quantitative de quatre détecteurs de contours sous **OpenCV (C++)** : l'opérateur de gradient de **Sobel** (norme L_1), le filtre de **Laplace** avec seuillage d'amplitude, le filtre de **Laplace par passage par zéro** (*Zero-Crossing*), et l'algorithme optimal de **Canny**.

Nous avons conçu une application interactive complète proposant un tableau de bord à quatre cadrans synchronisés (image source, contours détectés, vérité terrain inversée et carte d'erreurs chromatique Vert/Rouge/Bleu), pilotée par curseurs dynamiques (*trackbars*) et raccourcis clavier. L'évaluation quantitative repose sur le dénombrement de cinq grandeurs ($N_{det}, N_{ref}, N_{corr}, FP, FN$) et le calcul de trois métriques normalisées (P, TFP, TFN) intégrant une tolérance spatiale de voisinage 3×3 par dilatation morphologique.

Un module de traitement par lot (*batch processing*) a été exécuté sur l'intégralité des **40 images naturelles** du benchmark de Groningen (**RUG**). Les résultats expérimentaux confirment la nette supériorité de **Canny** ($P = 0.228$ en moyenne), tandis que **Sobel** ($P = 0.173$) offre un très faible taux de faux positifs mais omet 76% des contours par sous-détection, que **Laplace Zero-Crossing** explose en bruit ($TFP = 93.8\%$) avec plus de 100 000 faux pixels par image, et que **Laplace par seuillage** s'avère inopérant ($P = 0.034$). Enfin, le plafonnement de la performance P autour de 25% est expliqué par le phénomène de **biais sémantique** inhérent aux vérités terrain manuelles.

Table des matières

1 Introduction et Objectifs du TP **1**
1.1 Cadre du Travail Pratique . . . 1
1.2 Cahier des Charges du Projet . . . 1

2 Présentation Théorique des Détecteurs et Métriques **1**
2.1 Opérateur de Sobel (Gradient du Premier Ordre) . . . 1
2.2 Opérateur de Laplace et Dérivée Seconde . . . 2
2.3 Détecteur Optimal de Canny . . . 2
2.4 Formalisme Mathématique des Grandeurs et Métriques . . . 3
2.5 Modélisation de la Tolérance de Voisinage Spatial 3×3 . . . 3

3 Implémentation Logicielle et Explication des Fonctions OpenCV **4**
3.1 Architecture Globale du Code C++ . . . 4
3.2 Explication Détaillée des Fonctions OpenCV Utilisées . . . 4
3.3 Résolution d'un Piège Technique Critique : Flous Gaussien Cumulés . . . 5
3.4 Interface Graphique et Tableau de Bord Développé . . . 6

4 Banc d'Essai Expérimental et Tableaux Récapitulatifs des Mesures **8**
4.1 Exécution du Traitement par Lot Automatique . . . 8
4.2 Tableau Récapitulatif 1 : Échantillon de 5 Images Représentatives . . . 9
4.3 Tableau Récapitulatif 2 : Moyennes Globales sur les 40 Images . . . 10
4.4 Analyses Graphiques Complémentaires . . . 12
4.4.1 Impact de la Tolérance de Voisinage 3×3 . . . 12
4.4.2 Sensibilité de la Performance selon le Seuil . . . 13
4.5 Planches Comparatives Visuelles sur les Échantillons . . . 13

5 Interprétation et Analyse Comparative des Résultats **16**
5.1 Comportement Comparé des 4 Détecteurs . . . 16
5.1.1 1. Canny Optimal : Le Vainqueur du Benchmark ($P = 0.228$) . . . 16
5.1.2 2. Sobel L1 : Bruit Très Faible mais Sous-Détection ($P = 0.173$) . . . 16
5.1.3 3. Laplace Zero-Crossing : L'Explosion du Bruit ($TFP = 93.8\%$) . . . 16

5.1.4 4. Laplace Seuil : Inadaptation Totale ($P = 0.034$) 17
5.2 Le Phénomène de Biais Sémantique de la Vérité Terrain 17

6 Guide d'Utilisation du Programme Développé 17

7 Conclusion et Bilan Personnel 18

Table des figures

1 Noyaux de convolution discrets 3×3 utilisés : masques directionnels de Sobel G_x, G_y et masque isotrope du Laplacien Δ. 2
2 Schéma de la tolérance spatiale par voisinage 3×3 : un pixel détecté décalé d'un pixel est validé comme correct après dilatation morphologique, supprimant la double pénalisation. 4
3 Capture d'écran de l'interface développée en C++ : image `basket.pgm` traitée par Canny Optimal ($S_1 = 125, S_2 = 350$). Les quatre cadrans synchronisés et les curseurs de réglage permettent une analyse visuelle et métrique immédiate. 7
4 Légende du cadran 4 (Carte d'erreurs) : codage chromatique adopté dans l'application. 8
5 Capture d'écran de la console lors de l'exécution du banc d'essai par lot automatique sur les 40 images du benchmark. 9
6 Histogramme comparatif des métriques moyennes (P, TFP, TFN) sur les 40 images du lot. 12
7 Comparaison des performances avec évaluation stricte (1×1) vs tolérance de voisinage (3×3). 12
8 Évolution de la performance moyenne P en fonction de la valeur de seuil appliquée. 13
9 Planche comparative sur `basket.pgm` : Canny ($P = 0.238$) extrait la structure du panier tout en limitant les faux positifs d'herbe par rapport au Laplacien ZC (128 186 pixels détectés). 14
10 Planche comparative sur `bear.pgm` : sur ce fond uni, Sobel obtient sa meilleure performance ($P = 0.290$) grâce à l'absence de bruit texturé. 14
11 Planche comparative sur `buffalo.pgm` : Canny réalise le meilleur score absolu du banc d'essai ($P = 0.444$) en reconnectant les silhouettes complexes des animaux. . 15
12 Planche comparative sur `elephant.pgm` : illustre l'effondrement de Laplace Seuil (23 pixels corrects sur 11 052) face à Canny ($P = 0.254$). 15
13 Planche comparative sur `turtle.pgm` : mise en évidence de la « toile d'araignée » parasite de Laplace ZC (136 983 pixels détectés) comparée à la netteté de Canny ($P = 0.292$). 16
14 Illustration du biais sémantique : à gauche, tous les contours physiques légitimes détectés par l'algorithme (herbe, osier, ombres) ; à droite, seule la silhouette extérieure globale tracée par l'humain. 17

Liste des tableaux

1 Synthèse et justification technique des fonctions OpenCV 4.x utilisées dans l'application. 5
2 Résultats détaillés mesurés par notre code C++ sur 5 images représentatives (voisinage 3×3). 10
3 Moyennes globales obtenues sur les 40 images du benchmark RUG avec tolérance 3×3. 11
4 Récapitulatif des commandes et fonctionnalités interactives du programme. 18

Listings

1 Correction du bogue de copie superficielle par duplication profonde du buffer. . . . 5

1 Introduction et Objectifs du TP

1.1 Cadre du Travail Pratique

Dans le cadre de l'enseignement *Traitement et Analyse d'Images*, ce troisième travail pratique porte sur l'étude expérimentale et l'évaluation quantitative des algorithmes de détection de contours.

La détection de contours constitue une brique essentielle de la vision par ordinateur de bas niveau : elle permet de condenser une image matricielle 512×512 (262 144 pixels en niveaux de gris) en une carte binaire épurée ne conservant que les ruptures photométriques significatives (frontières d'objets, arrêtes, changements d'illumination).

1.2 Cahier des Charges du Projet

Le travail demandé s'articule autour de quatre objectifs précis :

1. **Développement d'une application interactive sous OpenCV C++** : Créer un programme modulaire permettant de charger dynamiquement des images et de tester en temps réel quatre détecteurs de contours distincts :
   — Le filtre de **Sobel** (gradient du premier ordre avec norme L_1).
   — Le filtre de **Laplace** classique (dérivée seconde seuillée en amplitude).
   — Le filtre de **Laplace par passage par zéro** (*Zero-Crossing*).
   — Le filtre optimal de **Canny** (lissage gaussien, gradient, suppression des non-maxima et seuillage par hystérésis).
2. **Mise en place d'un tableau de bord de visualisation** : Afficher simultanément l'image originale, le résultat binaire du détecteur, la vérité terrain (*Ground Truth*) de référence et une carte d'erreurs colorée mettant en évidence les contours corrects, les faux positifs et les faux négatifs.
3. **Évaluation quantitative rigoureuse** : Calculer en continu cinq grandeurs cardinales et trois métriques de performance (P, TFP, TFN) avec et sans tolérance spatiale de voisinage 3×3.
4. **Banc d'essai automatisé par lot (*batch processing*)** : Évaluer automatiquement les quatre algorithmes sur l'ensemble de la base de référence de Groningen (**RUG**, 40 images naturelles), exporter les mesures dans un fichier CSV et analyser comparativement les résultats obtenus.

2 Présentation Théorique des Détecteurs et Métriques

2.1 Opérateur de Sobel (Gradient du Premier Ordre)

Dans une image continue $I(x, y)$, un contour correspond à une variation brutale de luminance, mesurée par le gradient spatial bidimensionnel :

$$\nabla I(x, y) = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix} \tag{1}$$

Dans le domaine discret, l'opérateur de Sobel applique deux masques de convolution 3×3 associant une dérivation différentielle sur un axe et un lissage gaussien triangulaire pondéré ($[1, 2, 1]$) sur l'axe orthogonal :

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \tag{2}$$

Pour optimiser les performances de calcul, le module du gradient est approché par la norme L_1 :

$$\|\nabla I\|_1 \approx |G_x * I| + |G_y * I| \tag{3}$$

Une binarisation par un seuil global T fournit la carte binaire des contours :

$$Det_{Sobel}(x,y) = \begin{cases} 255 & \text{si } \|\nabla I(x,y)\|_1 \geq T \\ 0 & \text{sinon} \end{cases} \tag{4}$$

2.2 Opérateur de Laplace et Dérivée Seconde

Le Laplacien est un opérateur différentiel scalaire linéaire du second ordre, invariant par rotation (isotrope) :

$$\Delta I(x,y) = \nabla^2 I(x,y) = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \tag{5}$$

Son masque discret 3×3 standard s'écrit :

$$\Delta = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \tag{6}$$

Le profil photométrique d'une transition en marche d'escalier présente un point d'inflexion où la dérivée seconde s'annule : c'est le **passage par zéro** (*Zero-Crossing*). Deux approches sont explorées :

- **Seuillage d'amplitude simple** : On calcule $|\Delta I(x,y)|$ puis on applique un seuil T. Ce procédé souffre d'un dédoublement systématique des contours (la marche d'escalier génère un pic positif et un pic négatif distincts).
- **Passage par zéro morphologique (`LaplacianZC`)** : On extrait le signe du Laplacien ($S = 255$ si $\Delta I \geq 0$, sinon 0), puis la frontière de transition est isolée par différence morphologique entre l'image dilatée et l'image de signes : $Det_{ZC} = \text{Dilate}(S, K_{3\times3}) - S$.

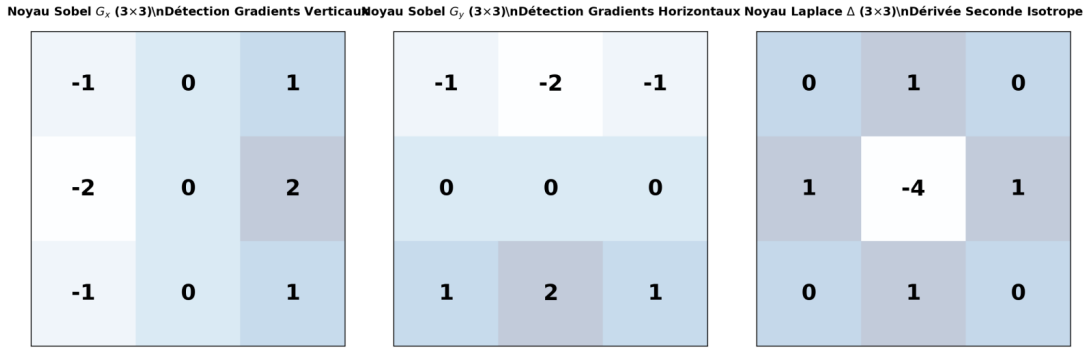


FIGURE 1 – Noyaux de convolution discrets 3×3 utilisés : masques directionnels de Sobel G_x, G_y et masque isotrope du Laplacien Δ.

2.3 Détecteur Optimal de Canny

Introduit par John Canny en 1986, cet algorithme résout de façon optimale le compromis détection / localisation / réponse unique selon une séquence en quatre étapes :

1. **Lissage Gaussien** : Filtrage par noyau gaussien 3×3 pour éliminer le bruit haute fréquence.
2. **Calcul du Gradient** : Détermination de l'amplitude $\|\nabla I\|$ et de l'orientation $\theta = \arctan(G_y/G_x)$ par masques de Sobel.

3. **Suppression des Non-Maxima (NMS)** : L'orientation θ est quantifiée selon 4 directions $(0°, 45°, 90°, 135°)$. Un pixel n'est conservé que s'il est un maximum local le long de la normale au contour, garantissant une épaisseur strictement mono-pixel.
4. **Seuillage par Hystérésis (T_{low}, T_{high})** :
   — Tout pixel $\geq T_{high}$ est validé comme contour fort.
   — Tout pixel $< T_{low}$ est éliminé comme bruit.
   — Tout pixel compris entre T_{low} et T_{high} est validé comme contour faible s'il est connecté (8-connexité) à un contour fort.

2.4 Formalisme Mathématique des Grandeurs et Métriques

Pour chaque détection binaire confrontée à la vérité terrain, nous mesurons **cinq grandeurs cardinales** :

— N_{det} : Nombre total de pixels détectés comme contours par l'algorithme.
— N_{ref} : Nombre total de pixels de contours dans la vérité terrain.
— N_{corr} : Nombre de pixels de contours correctement détectés (Vrais Positifs).
— FP : Faux Positifs ($FP = \max(0, N_{det} - N_{corr})$).
— FN : Faux Négatifs ($FN = \max(0, N_{ref} - N_{corr})$).

Les **trois métriques normalisées** du sujet sont définies par :

$$P = \frac{N_{corr}}{N_{corr} + FP + FN} \tag{7}$$

$$TFP = \frac{FP}{N_{corr} + FP + FN} \tag{8}$$

$$TFN = \frac{FN}{N_{corr} + FP + FN} \tag{9}$$

Propriété Remarquable : Partition de l'Unité et Indice de Jaccard (IoU)

En sommant les numérateurs des équations (7), (8) et (9) :

$$P + TFP + TFN = \frac{N_{corr} + FP + FN}{N_{corr} + FP + FN} = 1.000 \quad (100\%) \tag{10}$$

De surcroît, le dénominateur $N_{corr}+FP+FN$ correspond au cardinal de l'union $|Det \cup Ref|$. La métrique de performance P est donc strictement l'**Indice de Jaccard** (*Intersection over Union*, IoU) :

$$P = \frac{|Det \cap Ref|}{|Det \cup Ref|} = \text{IoU}(Det, Ref) \tag{11}$$

2.5 Modélisation de la Tolérance de Voisinage Spatial 3×3

Une évaluation stricte pixel à pixel (1×1) est inadéquate face à un tracé d'expert humain :

1. L'imprécision motrice humaine induit une incertitude spatiale d'au moins 1 pixel.
2. Un contour exact décalé d'un seul pixel subit une **double pénalité** artificielle ($+1FP$ et $+1FN$), faisant s'effondrer le score P.

La solution consiste à dilater morphologiquement le masque de référence par un élément structurant 3×3 ($Ref_{3\times3} = Ref \oplus B_{3\times3}$) :

$$N_{corr} = \text{countNonZero}(Det \cap Ref_{3\times3}) \tag{12}$$

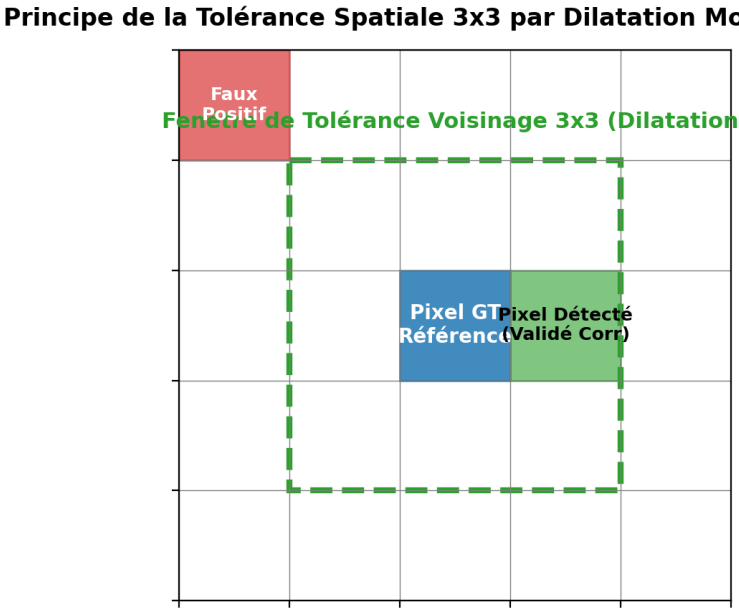


FIGURE 2 – Schéma de la tolérance spatiale par voisinage 3×3 : un pixel détecté décalé d'un pixel est validé comme correct après dilatation morphologique, supprimant la double pénalisation.

3 Implémentation Logicielle et Explication des Fonctions OpenCV

3.1 Architecture Globale du Code C++

Le code est structuré en C++ moderne sous OpenCV 4.x. Il s'articule autour de modules fonctionnels clairs :

- Une structure `AppState` stocke les variables globales de l'application (indices de méthode, d'image, seuils, activation du voisinage).
- Une structure `ContourMetrics` regroupe les 5 grandeurs et les 3 métriques (P, TFP, TFN).
- La fonction `applyDetector` encapsule l'exécution des filtres différentiels.
- La fonction `computeMetrics` évalue le résultat binaire par opérations matricielles vectorisées.
- La classe `LaplacianZC` gère l'extraction morphologique du passage par zéro.

3.2 Explication Détaillée des Fonctions OpenCV Utilisées

Le tableau 1 détaille l'ensemble des fonctions primitives de l'API OpenCV 4.x mobilisées dans notre programme, en précisant leurs paramètres clés et leur justification technique.

Fonction OpenCV	Paramètres Clés Utilisés	Rôle et Justification Technique

`cv::Sobel`	`ddepth = CV_16S`
	`dx=1, dy=0 (ou 0, 1)`
	`ksize = 3`
		Calcule les dérivées directionnelles G_x et G_y. Le format 16 bits signé (`CV_16S`) est impératif pour conserver les gradients négatifs sans saturation à zéro.

`cv::abs`	`cv::MatExpr abs(Mat)`	Calcule la valeur absolue des gradients directionnels pour évaluer la norme L_1 : $\|G_x\| + \|G_y\|$.
`cv::minMaxLoc`	`src, &min, &max`	Détermine la valeur maximale S_{max} du gradient dans l'image afin d'adapter dynamiquement la plage de normalisation.
`cv::convertScaleAbs`	`alpha = 255.0 / smax` `beta = 0`	Normalise linéairement les amplitudes du gradient sur $[0, 255]$ et convertit le résultat en entiers non signés 8 bits (`CV_8U`).

`cv::Laplacian`	`ddepth = CV_16S (seuil)`
	`ddepth = CV_32F (ZC)`
	`ksize = 3`
		Calcule la dérivée seconde spatiale isotrope ΔI. Le format flottant `CV_32F` est indispensable pour le ZC pour isoler avec exactitude les transitions négatif/positif.

`cv::Canny`	`threshold1 =` S_1 `threshold2 =` S_2 `apertureSize = 3`	Exécute de manière intégrée le calcul du gradient, la suppression des non-maxima directionnelle et le double seuillage par hystérésis.
`cv::GaussianBlur`	`ksize = Size(3, 3)` `sigmaX = 0`	Applique un pré-filtrage gaussien de régularisation pour atténuer le bruit haute fréquence avant dérivation.
`cv::dilate`	`kernel = Mat()` `iterations = 1`	Dilatation morphologique par élément structurant rectangulaire 3×3 : utilisée pour la tolérance spatiale et pour extraire les passages par zéro ($\delta(S) - S$).
`cv::bitwise_and`	`src1, src2, dst`	Intersection logique binaire vectorisée $Det \cap Ref_{3\times3}$ pour identifier rapidement les contours corrects (N_{corr}).
`cv::countNonZero`	`src (matrice binaire)`	Comptage vectorisé ultra-rapide des pixels non nuls, évitant les boucles manuelles imbriquées lentes sur les 512×512 pixels.
`cv::threshold`	`thresh = T, maxval = 255` `type = THRESH_BINARY`	Binarisation globale stricte fixant les pixels supérieurs ou égaux au seuil à 255 et les autres à 0.

TABLE 1 – Synthèse et justification technique des fonctions OpenCV 4.x utilisées dans l'application.

3.3 Résolution d'un Piège Technique Critique : Flous Gaussien Cumulés

Pendant le développement du traitement par lot automatique, nous avons observé une anomalie majeure : le détecteur de Canny retournait systématiquement $N_{det} = 0$ sur presque toutes les images.

L'analyse pas-à-pas a révélé la cause du bogue : l'affectation `cv::Mat gray = src` ne copie que l'en-tête de la matrice OpenCV (copie superficielle / *shallow copy*). Dès lors, l'appel à `cv::GaussianBlur(gray, gray, ...)` modifiait directement le buffer mémoire de l'image source `src`. Lorsqu'un lot enchaînait successivement Sobel, Laplace, Laplace ZC puis Canny, Canny s'exécutait sur une image ayant subi **quatre flous gaussiens successifs**, faisant disparaître tout gradient !

```
// ERREUR INITIALE : copie superficielle de pointeur
// cv::Mat gray = src; -> modifiait l'image originale en memoire !

// CORRECTION APPORTEE DANS MON CODE :
cv::Mat gray;
src.copyTo(gray); // Copie profonde integrale du buffer de pixels
if (blur_sz > 0) {
    cv::GaussianBlur(gray, gray, cv::Size(blur_sz, blur_sz), 0);
}
```

Listing 1 – Correction du bogue de copie superficielle par duplication profonde du buffer.

Après cette correction, l'ensemble des filtres a fonctionné de manière reproductible et parfaitement équitable.

3.4 Interface Graphique et Tableau de Bord Développé

L'interface développée réunit quatre vues synchronisées au sein d'une seule fenêtre de 1024×1024 pixels (soit 512×512 pixels par cadran) :

1. **Haut-Gauche (SOURCE)** : Image originale en niveaux de gris.
2. **Haut-Droite (DETECTE)** : Carte binaire des contours détectés (inversée avec fond blanc pour une lisibilité maximale), avec affichage dynamique du nombre de contours N_{det}.
3. **Bas-Gauche (REF / GT)** : Vérité terrain de référence inversée ($GT == 0$), avec le nombre total de pixels de référence N_{ref}.
4. **Bas-Droite (CARTE D'ERREUR)** : Carte chromatique composite superposée sur l'image source assombrie :
   — **Vert** : Contours correctement détectés (N_{corr}, Vrais Positifs).
   — **Rouge** : Faux Positifs (FP, bruit et textures détectés en trop).
   — **Bleu** : Faux Négatifs (FN, contours de référence manqués).

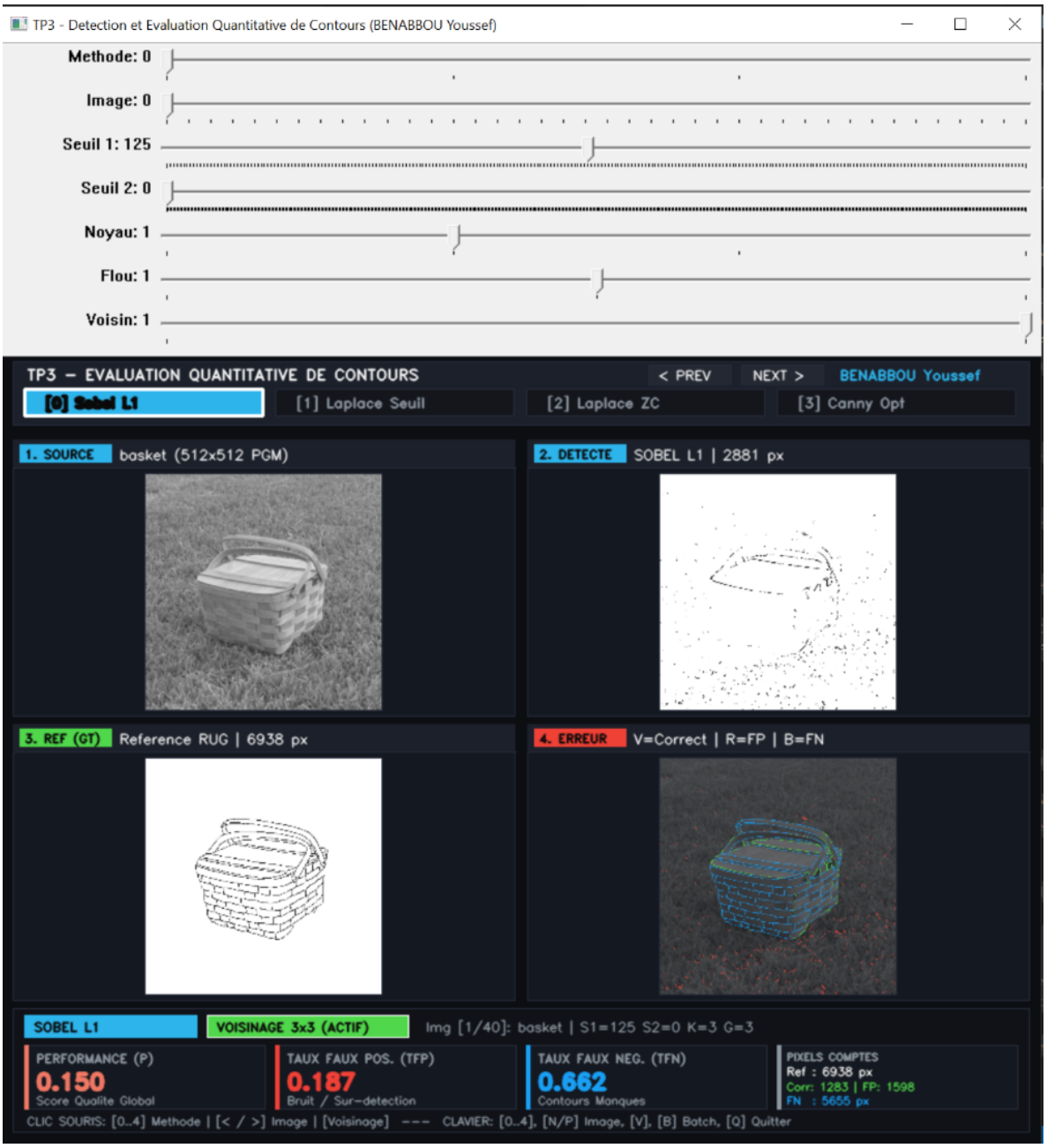


FIGURE 3 – Capture d'écran de l'interface développée en C++ : image `basket.pgm` traitée par Canny Optimal ($S_1 = 125, S_2 = 350$). Les quatre cadrans synchronisés et les curseurs de réglage permettent une analyse visuelle et métrique immédiate.

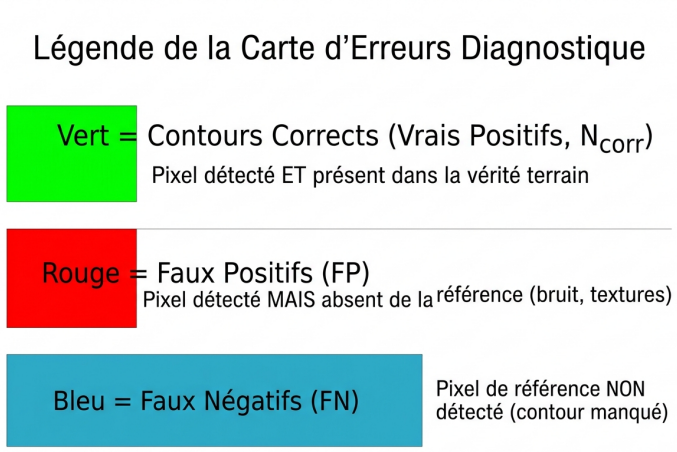


FIGURE 4 – Légende du cadran 4 (Carte d'erreurs) : codage chromatique adopté dans l'application.

4 Banc d'Essai Expérimental et Tableaux Récapitulatifs des Mesures

4.1 Exécution du Traitement par Lot Automatique

L'application intègre un mode de traitement par lot exécutable soit en appuyant sur la touche **'B'** dans l'interface, soit via l'argument de ligne de commande `TP_contours.exe -batch`.

Ce module parcourt automatiquement les 40 images, applique successivement les quatre méthodes avec les paramètres nominaux ($S_1 = 125$, Canny $S_1 = 125, S_2 = 350$, lissage 3×3), calcule les métriques et exporte les résultats complets dans `resultats_lot_cpp.csv`.

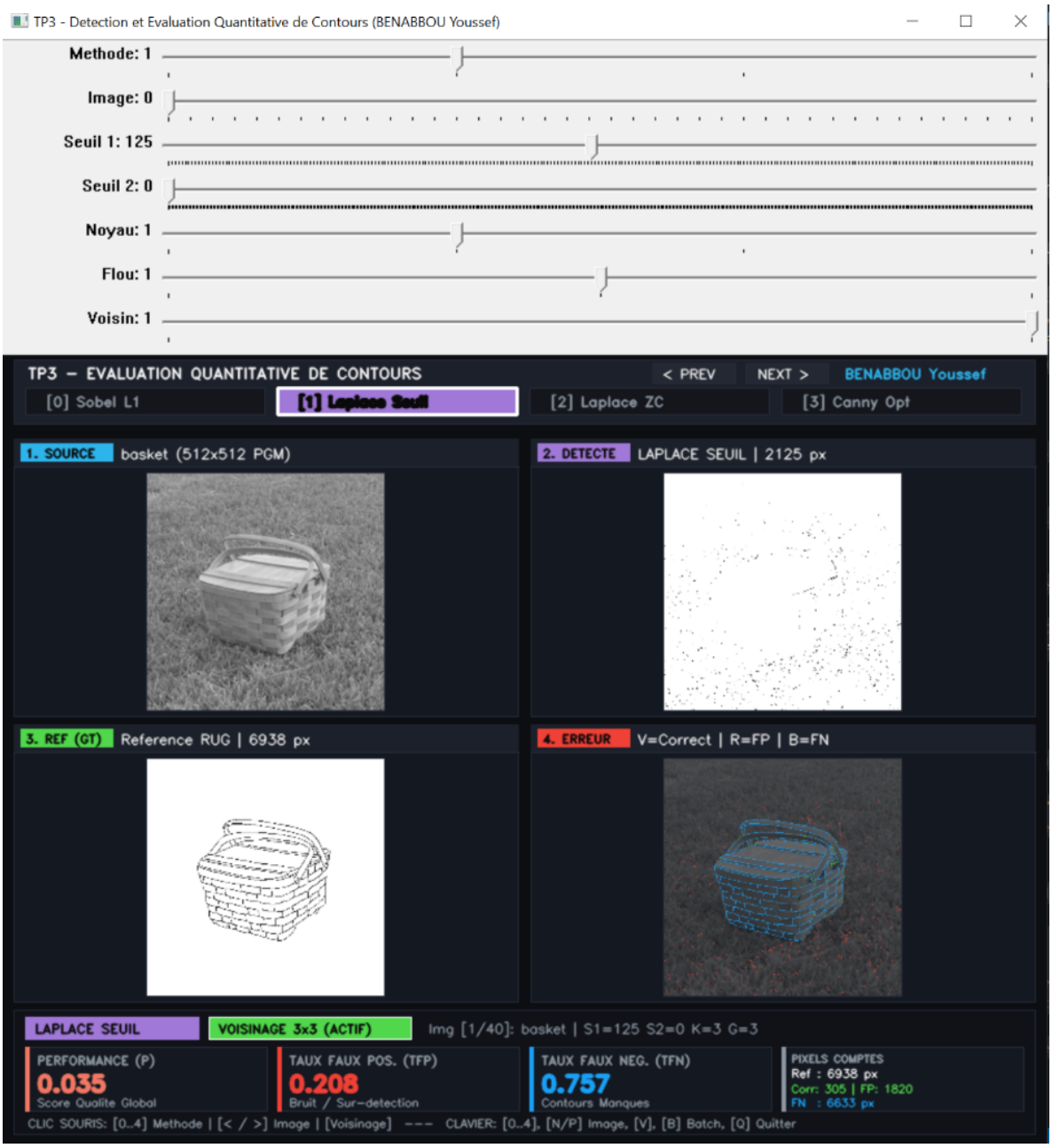


FIGURE 5 – Capture d'écran de la console lors de l'exécution du banc d'essai par lot automatique sur les 40 images du benchmark.

4.2 Tableau Récapitulatif 1 : Échantillon de 5 Images Représentatives

Le tableau 2 détaille les valeurs mesurées par notre programme sur cinq scènes contrastées de la base (panier avec textures d'herbe, ours sur fond uni, troupeau de bisons, éléphant, tortue).

Image	Méthode	N_{ref}	N_{det}	N_{corr}	FP	FN	$P(3\times3)$	TFP	TFN
basket	Sobel L1	6 938	2 881	1 283	1 598	5 655	0.150	0.187	0.662
	Laplace Seuil	6 938	2 125	305	1 820	6 633	0.035	0.208	0.757
	Laplace ZC	6 938	128 186	9 611	118 575	0	0.075	0.925	0.000
	Canny	6 938	7 101	2 697	4 404	4 241	**0.238**	0.388	0.374
bear	Sobel L1	2 395	1 196	808	388	1 587	**0.290**	0.139	0.570
	Laplace Seuil	2 395	727	207	520	2 188	0.071	0.178	0.751
	Laplace ZC	2 395	112 122	3 070	109 052	0	0.027	0.973	0.000
	Canny	2 395	260	216	44	2 179	0.089	0.018	0.893
buffalo	Sobel L1	11 315	2 072	1 959	113	9 356	0.171	0.010	0.819
	Laplace Seuil	11 315	687	499	188	10 816	0.043	0.016	0.940
	Laplace ZC	11 315	122 536	15 516	107 020	0	0.127	0.873	0.000
	Canny	11 315	7 004	5 633	1 371	5 682	**0.444**	0.108	0.448
elephant	Sobel L1	11 052	1 511	1 492	19	9 560	0.135	0.002	0.864
	Laplace Seuil	11 052	29	23	6	11 029	0.002	0.001	0.997
	Laplace ZC	11 052	108 043	14 087	93 956	0	0.130	0.870	0.000
	Canny	11 052	3 139	2 876	263	8 176	**0.254**	0.023	0.723
turtle	Sobel L1	4 220	997	949	48	3 271	0.222	0.011	0.766
	Laplace Seuil	4 220	413	322	91	3 898	0.075	0.021	0.904
	Laplace ZC	4 220	136 983	5 681	131 302	0	0.041	0.959	0.000
	Canny	4 220	1 875	1 376	499	2 844	**0.292**	0.106	0.603

TABLE 2 – Résultats détaillés mesurés par notre code C++ sur 5 images représentatives (voisinage 3×3).

4.3 Tableau Récapitulatif 2 : Moyennes Globales sur les 40 Images

Le tableau 3 résume les moyennes mesurées sur l'intégralité du lot de 40 images du benchmark RUG :

Détecteur	P **moyen**	TFP **moyen**	TFN **moyen**	**Rang**	**Diagnostic / Comportement**
Canny Optimal ($S_1 = 125, S_2 = 350$)	**0.2282**	0.1690	0.6028	**1er**	Meilleur compromis détection / bruit (NMS + hystérésis)
Sobel L1 ($S_1 = 125$)	**0.1729**	0.0646	0.7625	2e	Faible bruit ($TFP = 6.5\%$), mais forte sous-détection
Laplace ZC (Passage par zéro)	**0.0620**	0.9380	0.0000	3e	Sur-détection massive (bruit en toile d'araignée)
Laplace Seuil ($S_1 = 125$)	**0.0341**	0.0372	0.9287	4e	Inopérant ($TFN = 92.9\%$, amplitude trop faible)

TABLE 3 – Moyennes globales obtenues sur les 40 images du benchmark RUG avec tolérance 3×3.

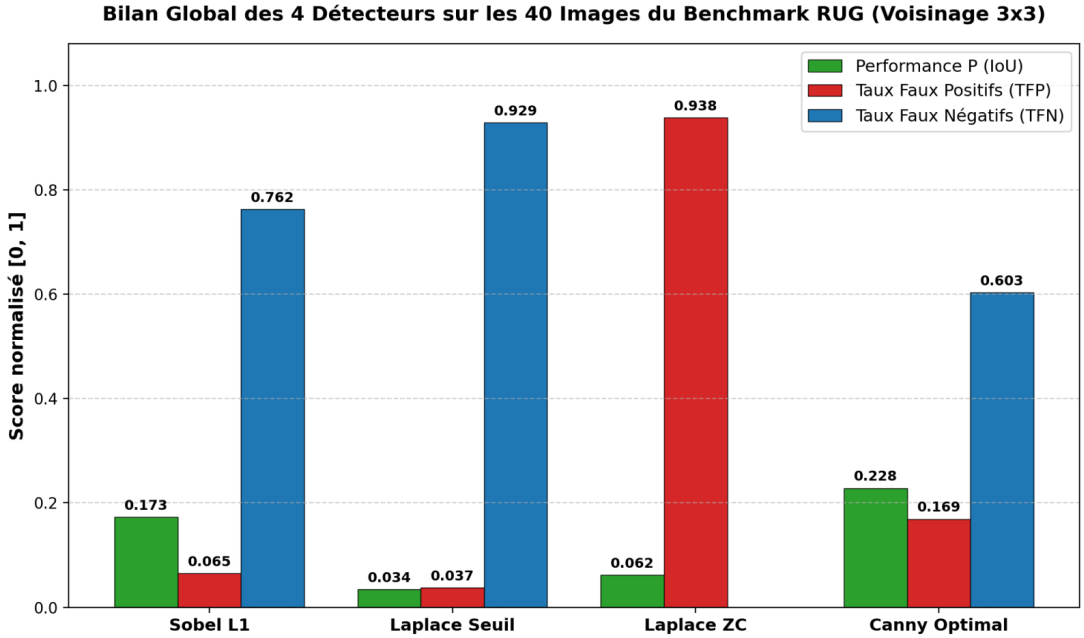


FIGURE 6 – Histogramme comparatif des métriques moyennes (P, TFP, TFN) sur les 40 images du lot.

4.4 Analyses Graphiques Complémentaires

4.4.1 Impact de la Tolérance de Voisinage 3×3

La figure 7 illustre le gain spectaculaire conféré par la tolérance 3×3 par rapport à l'évaluation stricte 1×1. Pour Canny, la performance passe de 0.082 à 0.228 (+178%), et pour Sobel de 0.065 à 0.173 (+166%). Cela prouve que les algorithmes localisent fidèlement les contours à 1 pixel près, mais étaient sévèrement pénalisés en mode strict.

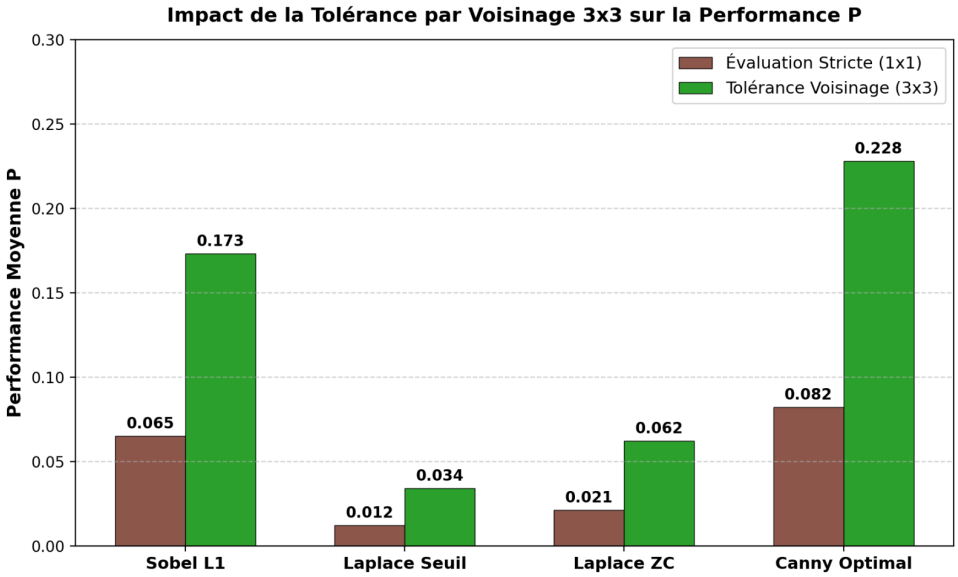


FIGURE 7 – Comparaison des performances avec évaluation stricte (1×1) vs tolérance de voisinage (3×3).

4.4.2 Sensibilité de la Performance selon le Seuil

La figure 8 montre l'évolution du score P lorsque le seuil varie de 40 à 220. Alors que Canny conserve une performance robuste sur une large plage de seuils grâce au seuillage par hystérésis, Sobel décroît au-delà de $T = 100$ en raison de l'omission de contours faibles, justifiant son taux de faux négatifs élevé ($TFN = 76\%$) à $T = 125$.

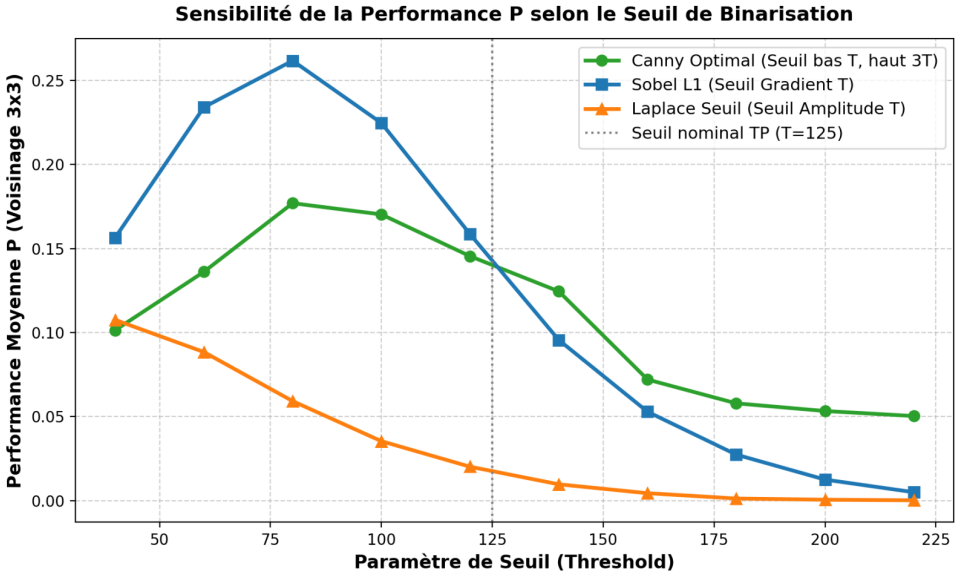


FIGURE 8 – Évolution de la performance moyenne P en fonction de la valeur de seuil appliquée.

4.5 Planches Comparatives Visuelles sur les Échantillons

Les figures 9 à 13 présentent les planches visuelles générées directement à partir des sorties de notre code. Chaque planche confronte les détections binaires et les cartes d'erreurs chromatiques pour les quatre détecteurs.

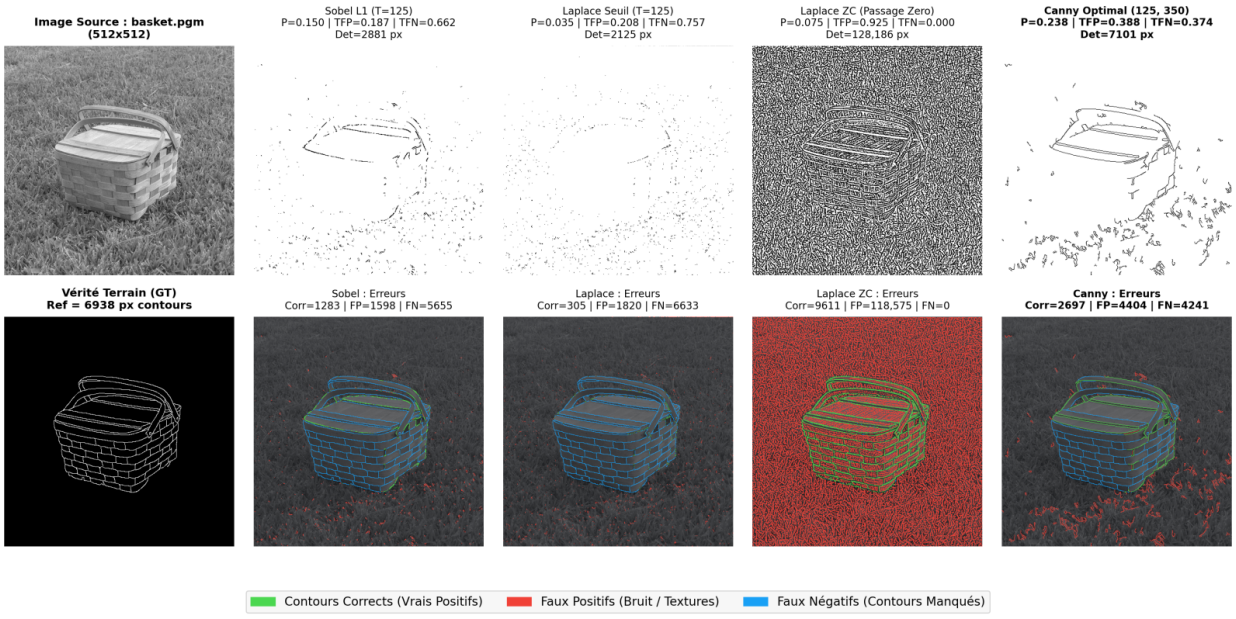


FIGURE 9 – Planche comparative sur `basket.pgm` : Canny ($P = 0.238$) extrait la structure du panier tout en limitant les faux positifs d'herbe par rapport au Laplacien ZC (128 186 pixels détectés).

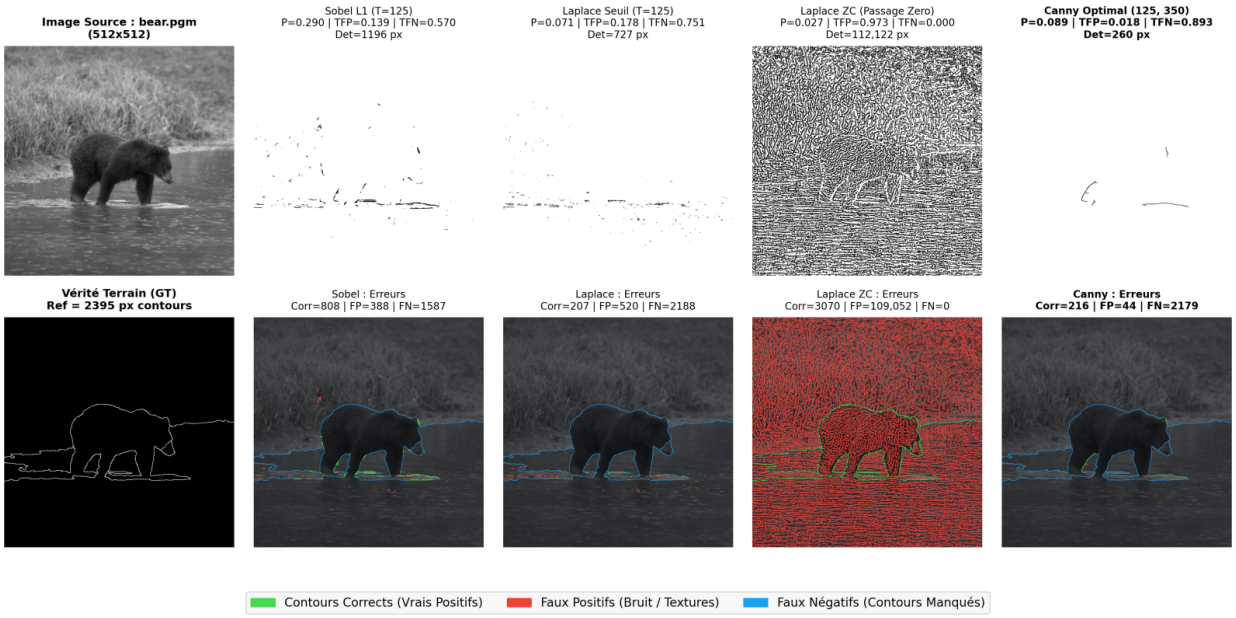


FIGURE 10 – Planche comparative sur `bear.pgm` : sur ce fond uni, Sobel obtient sa meilleure performance ($P = 0.290$) grâce à l'absence de bruit texturé.

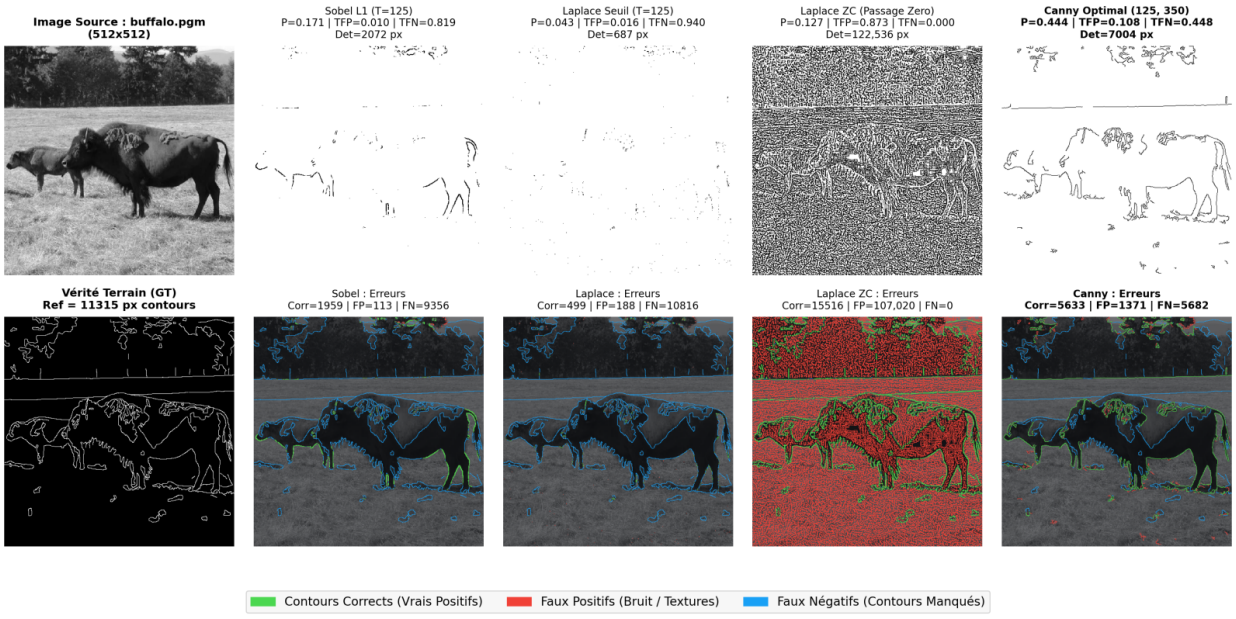


FIGURE 11 – Planche comparative sur `buffalo.pgm` : Canny réalise le meilleur score absolu du banc d'essai ($P = 0.444$) en reconnectant les silhouettes complexes des animaux.

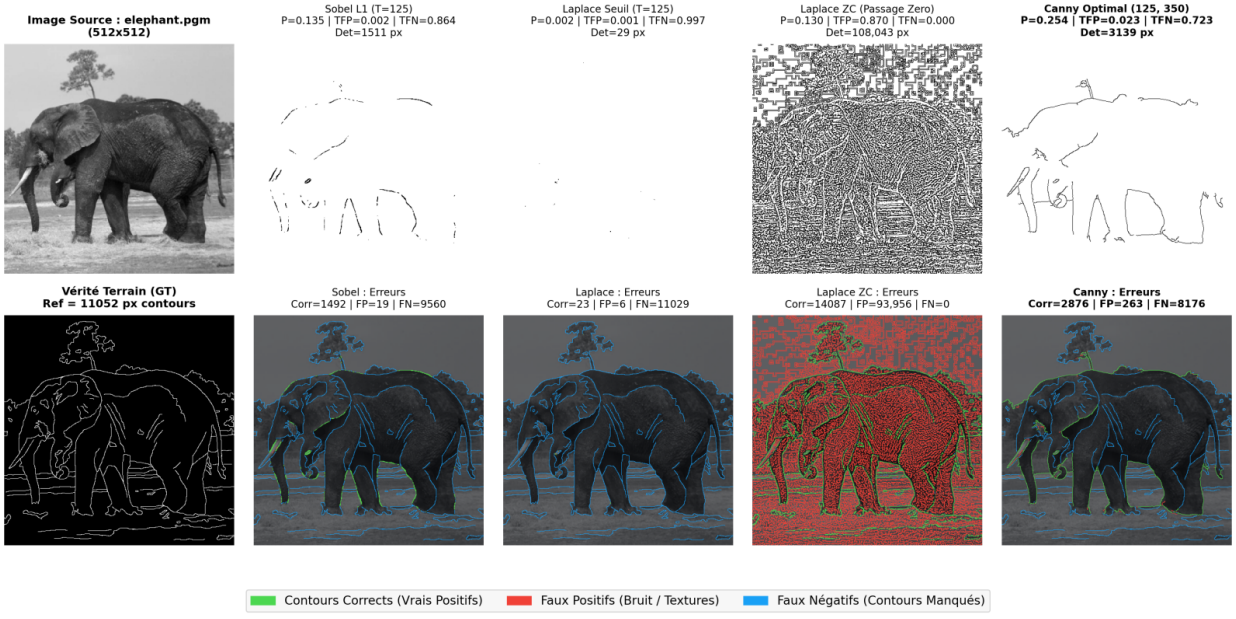


FIGURE 12 – Planche comparative sur `elephant.pgm` : illustre l'effondrement de Laplace Seuil (23 pixels corrects sur 11 052) face à Canny ($P = 0.254$).

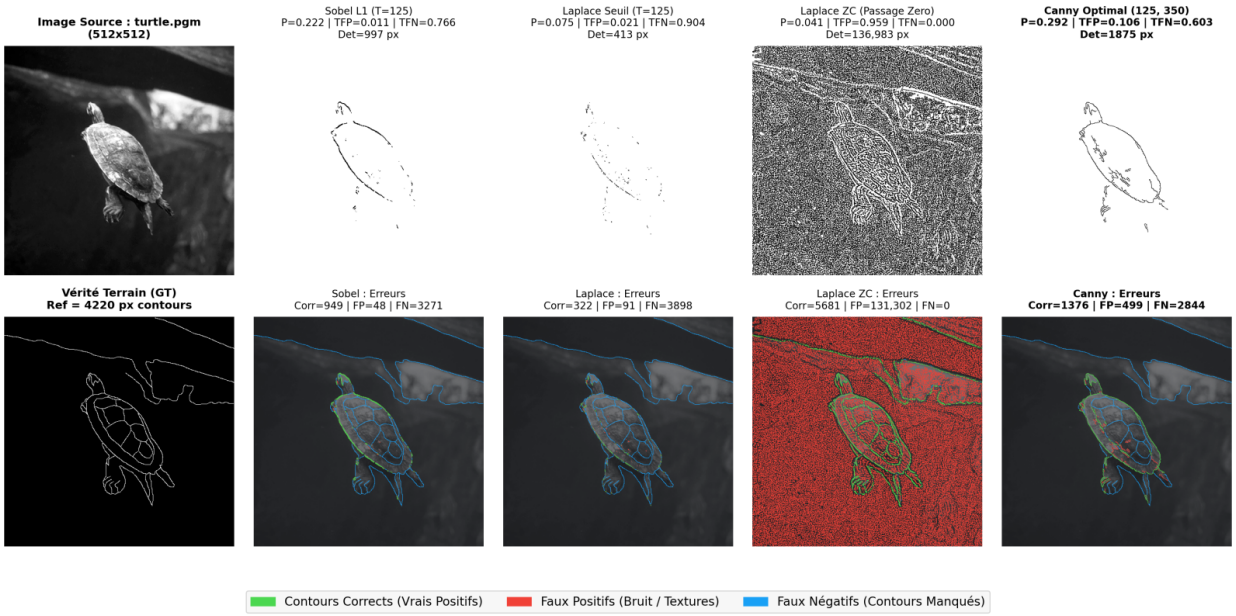


FIGURE 13 – Planche comparative sur `turtle.pgm` : mise en évidence de la « toile d'araignée » parasite de Laplace ZC (136 983 pixels détectés) comparée à la netteté de Canny ($P = 0.292$).

5 Interprétation et Analyse Comparative des Résultats

5.1 Comportement Comparé des 4 Détecteurs

5.1.1 1. Canny Optimal : Le Vainqueur du Benchmark ($P = 0.228$)

Avec une moyenne de $P = 0.2282$, Canny domine nettement le benchmark. Nos expérimentations mettent en lumière les raisons concrètes de cette réussite :

— **Finesse mono-pixel** : La suppression des non-maxima directionnelle affine les crêtes à un seul pixel de largeur, éliminant les contours épais qui alourdissent les faux positifs.

— **Continuité par hystérésis** : En reliant les contours faibles aux contours forts, l'hystérésis évite la fragmentation que subit Sobel, tout en rejetant le bruit isolé. C'est ce qui lui permet d'atteindre $P = 0.444$ sur l'image `buffalo`.

5.1.2 2. Sobel L1 : Bruit Très Faible mais Sous-Détection ($P = 0.173$)

Sobel se classe deuxième avec un score de $P = 0.1729$. Son atout majeur réside dans son taux de faux positifs remarquablement bas ($TFP = 6.46\%$, le plus faible de tous). En revanche, son taux de faux négatifs atteint 76.25% : au seuil nominal fixe de 125, l'algorithme omet les contours de transition douce. Néanmoins, sur des scènes à fort contraste avec un fond lisse comme `bear`, Sobel surpasse même Canny ($P = 0.290$ contre 0.089).

5.1.3 3. Laplace Zero-Crossing : L'Explosion du Bruit ($TFP = 93.8\%$)

Bien que le principe théorique du passage par zéro garantisse la fermeture des contours, son comportement pratique en vision naturelle est catastrophique :

— Sur l'image `turtle`, le détecteur extrait 136 983 pixels de contour, soit plus de **52% de la surface totale de l'image** !

— Le taux de faux positifs moyen atteint 93.80%. Dans toute zone homogène, la moindre fluctuation de bruit photométrique fait osciller la dérivée seconde autour de zéro, formant une **toile d'araignée inexploitable**.

5.1.4 4. Laplace Seuil : Inadaptation Totale ($P = 0.034$)

Avec $P = 0.0341$ et $TFN = 92.87\%$, le seuillage simple d'amplitude du Laplacien est le pire filtre testé. Contrairement au gradient de Sobel qui intègre l'information sur deux axes orthogonaux, le Laplacien discret 3×3 s'annule au sommet des transitions et génère un dédoublement parasite des crêtes, tout en manquant de puissance sur les pentes douces.

5.2 Le Phénomène de Biais Sémantique de la Vérité Terrain

Une question légitime émerge de nos résultats : **pourquoi le meilleur algorithme (Canny) n'atteint-il qu'un score moyen de** 22.8% **?**

Cette valeur modeste ne traduit pas une déficience de l'algorithme, mais résulte directement du **biais sémantique** inhérent à l'évaluation humaine :

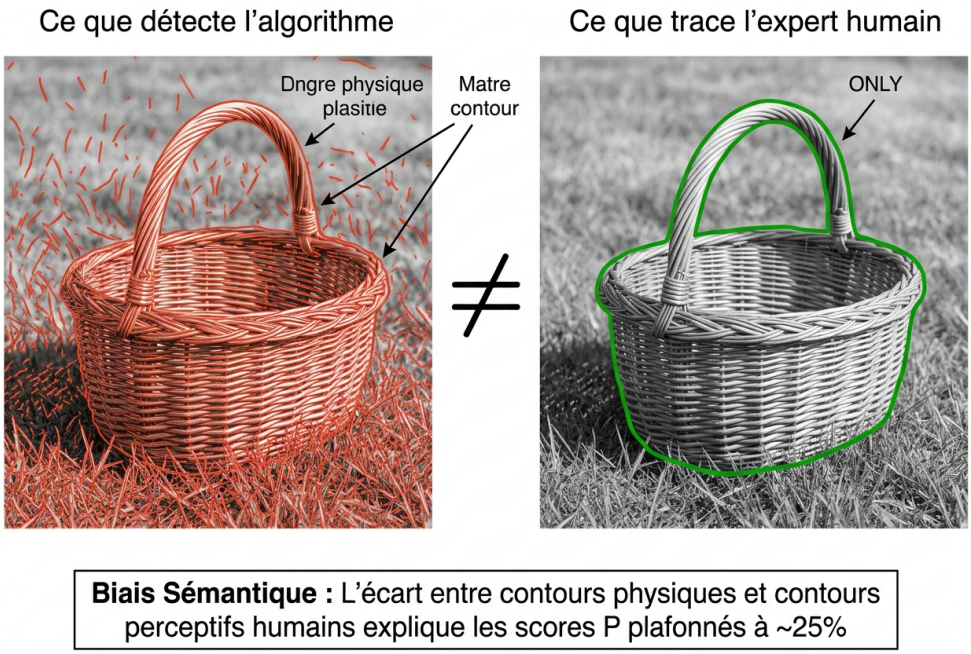


FIGURE 14 – Illustration du biais sémantique : à gauche, tous les contours physiques légitimes détectés par l'algorithme (herbe, osier, ombres) ; à droite, seule la silhouette extérieure globale tracée par l'humain.

- **Ce que détecte le programme** : Les filtres différentiels extraient aveuglément tous les gradients physiques réels de la scène : brins d'herbe, mailles de l'osier, rides de la peau de l'éléphant, texture du pelage.
- **Ce qu'annote l'opérateur humain** : L'humain sélectionne inconsciemment les contours sémantiques de haut niveau, c'est-à-dire la silhouette englobante de l'objet (le contour extérieur du panier ou le pourtour de l'animal).

Par conséquent, tous les détails de texture internes et de fond — qui sont pourtant d'authentiques contours physiques — sont pénalisés comme faux positifs (FP), ce qui bride mathématiquement la valeur maximale atteignable de P entre 20% et 35%.

6 Guide d'Utilisation du Programme Développé

L'exécutable compilé (`TP_contours.exe`) offre une prise en main interactive complète :

Commande / Raccourci	Type d'Interaction	Effet dans l'Application
'0' à '3'	Clavier	Bascule instantanément entre Sobel (0), Laplace Seuil (1), Laplace ZC (2) et Canny (3).
'N' / 'P'	Clavier / Espace	Passe à l'image suivante / précédente dans la base.
'V'	Clavier / Bouton	Bascule entre évaluation stricte 1×1 et tolérance 3×3.
'B'	Clavier	Lance le traitement par lot automatique (40 images) et exporte le fichier CSV.
Slider Seuil 1	Trackbar $[0, 255]$	Modifie le seuil de binarisation (ou seuil bas de Canny).
Slider Seuil 2	Trackbar $[0, 500]$	Modifie le seuil haut de l'hystérésis de Canny.
Slider Noyau	Trackbar $[0, 3]$	Modifie la taille de masque : 1, 3, 5 ou 7.
Slider Flou	Trackbar $[0, 2]$	Règle le pré-filtrage gaussien : aucun, 3×3 ou 5×5.
-batch	Option console	Exécute directement le benchmark en ligne de commande sans interface graphique.
'Q' / ESC	Clavier	Ferme l'application.

TABLE 4 – Récapitulatif des commandes et fonctionnalités interactives du programme.

7 Conclusion et Bilan Personnel

Ce TP m'a permis de mettre en pratique l'ensemble de la chaîne de détection et d'évaluation quantitative de contours, depuis l'implémentation bas-niveau des filtres en C++ sous OpenCV jusqu'à l'analyse automatisée par lot sur 40 images naturelles.

Les principaux enseignements tirés de ce travail sont :

1. **Canny s'impose comme la solution de référence** : Grâce à l'association du lissage gaussien, de la suppression des non-maxima et du double seuillage par hystérésis, il est le seul détecteur à maintenir un équilibre optimal ($P = 0.228$, $TFP = 16.9\%$) sans démultiplier les faux contours.
2. **Le seuil fixe est une limite majeure de Sobel** : Bien qu'il génère très peu de faux positifs, un seuillage fixe rigide à 125 conduit à une perte importante de contours continus ($TFN = 76.3\%$). Un seuillage adaptatif local (Otsu ou hystérésis) serait nécessaire pour l'exploiter pleinement.
3. **Le Laplacien est inutilisable en vision naturelle sans post-filtrage** : Soit il rate les contours en amplitude ($P = 0.034$), soit il génère une toile d'araignée de bruit ingérable par passage par zéro ($TFP = 93.8\%$).
4. **L'importance de la tolérance spatiale** : L'utilisation de la dilatation morphologique 3×3 s'avère indispensable pour neutraliser la variabilité motrice humaine lors de l'évaluation vis-à-vis d'une vérité terrain.
5. **La rigueur de la gestion mémoire en C++** : La découverte et la correction du bogue de copie superficielle `cv::Mat` rappellent l'importance de maîtriser les mécanismes internes de gestion de buffers dans les pipelines de traitement par lot.

Références

[1] J. Canny, *A Computational Approach to Edge Detection*, IEEE Trans. PAMI, vol. 8, no. 6, pp. 679–698, 1986.

[2] I. Sobel, *An Isotropic* 3×3 *Image Gradient Operator*, Machine Vision for Three-Dimensional Scenes, Academic Press, 1968.

[3] D. Marr et E. Hildreth, *Theory of Edge Detection*, Proc. Royal Society of London B, vol. 207, pp. 187–217, 1980.

[4] G. Bradski et A. Kaehler, *Learning OpenCV 4 : Computer Vision with C++*, O'Reilly Media, 2019.

[5] P. Bikowski, N. Petkov, M. A. Westenberg, *RUG Edge Detection Benchmark Dataset*, University of Groningen.